B. TECH CSE (INTERNET OF THINGS) SEMESTER-III

B. Tech -SEMESTER III

Course Code	Type	Course	L	T	P	Credits	Evaluation Scheme					Offering Dept.	Course Type	
							Theory			Practical			AICTE	NEP-2020
							CA	MS	ES	CA	ES			
CICPC301	CC	Software Engineering	3	0	2	4	-	20	50	30	-	CSE	Program Core	Discipline Specific
CICPC302	CC	Database Management Systems	3	0	2	4	-	20	50	30	-	CSE	Program Core	Discipline Specific
CICPC303	CC	Design and Analysis of Algorithms	3	0	2	4	-	20	50	30	-	CSE	Program Core	Discipline Specific
CICPC304	CC	Computer Architecture and Organization	3	1	0	4	30	20	50	-	-	CSE	Program Core	Discipline Specific
CIEPC305	CC	Microprocessor and Microcontroller s	3	0	2	4	-	20	50	30	-	ECE	Engg Sciences	Interdisciplinary
FCFO03xx	CES	Community Internship	-	-	-	2	100	-	-	-	-	-	Mandatory	Community Engagement and Service
VAXXxxx	VAC	--	-	-	-	NIL	100	-	-	-	-	-	Mandatory	VAC
		24 contact hours **				22								

** Actual teaching hours shall depend on LTP of all the courses

(Dr. R.S. Rao)

15/07/2024

(Dr. Vishal Gupta)

Forwarded for kind consideration & approval of BOS please.

Prof. B.K. Singh (Chairman BOS)

Course No	Title of the Course	Course Structure	Pre-Requisite
CICPC301	Software Engineering	3L - 0T - 2P	None

COURSE OUTCOMES (COs)

Students should be able to :-

- **CO 1 :** Understand the concepts of software engineering, software process and SDLC.
- **CO 2 :** Comprehend the concepts of software requirement, engineering process & software quality assurance.
- **CO 3 :** Illustrate the concepts of software design strategies, software measurements and metrics.
- **CO 4 :** Evaluate and compare different types of software testing to validate the project.
- **CO 5 :** Introduce and apply the software project management and their associated risks.

A. Syllabus

UNIT	CONTENTS
1.	**Introductionto Software Engineering:** The Evolving Role of Software, Changing Nature of Software, Software Engineering Discipline, Software Myths. **A Generic View of Process:** Software Engineering- A Layered Technology, A Process Framework, The Capability Maturity Model Integration (CMMI), Process Patterns, Process Assessment, Personal andTeam Process Models. **Software Development Life Cycle (SDLC)Models:** The Waterfall Model, Incremental Process Models, Evolutionary Process Models, Prototype Model, The Unified Process, Spiral Model, COCOMO Model
2.	**Software Requirements:** Functional and Non-Functional Requirements, User Requirements, System Requirements, Interface Specification, The Software Requirements Document. **Requirements Engineering Process:** Feasibility Studies, Requirements Elicitation and Analysis, Requirements Validation, Requirements Management, Data Flow Diagrams, Entity Relationship Diagrams. **Software Quality Assurance (SQA):** Verification and Validation, SQA Plans, Software Quality Frameworks, ISO 9000 Models, SEI-CMM Model.
3.	**Design Strategies:** Function Oriented Design, Object Oriented Design, Top-Down and Bottom-Up Design. **Software Measurement and Metrics:** Various Size Oriented Measures, Function Point (FP) Based Measures, Cyclomatic Complexity Measures, Control Flow Graphs.
4.	**Software Testing:** Testing Objectives, Unit Testing, Integration Testing, Acceptance Testing, Regression Testing, Testing for Functionality and Testing for Performance, Validation Testing, System Testing **Top Down and Bottom- UpTesting Strategies:** Test Drivers and Test Stubs, Structural Testing (White Box Testing), Functional Testing (Black Box Testing), Test Data Suit Preparation, Alpha and Beta Testing of Products. **Static Testing Strategies:** Formal Technical Reviews (Peer Reviews), Walk Through, Code Inspection, Compliance with Design and Coding Standards.
5.	**Software Project Management:** Project Scheduling, Staffing, Software Configuration Management, Quality Assurance, Project Monitoring **Risk Management:** Reactive Vs Proactive Risk Strategies, Software Risks, Risk Identification, RiskProjection, Risk Refinement, RMMM, RMMM Plan.

SUGGESTED READINGS:

1. Software Engineering, A practitioner's Approach- Roger S. Pressman, 6th edition. Mc GrawHill International Edition.
2. Software Engineering- Sommerville, 7th edition, Pearson Education.
3. Ghezzi, Software Engineering, PHI.
4. IEEE Standards on Software Engineering. Kane, Software Defect Prevention, SPD.
5. KK Aggarwal and Yogesh Singh, Software Engineering, New Age International Publishers.
6. Kassem Saleh, "Software Engineering", Cengage Learning.
7. Benmenachen, Software Quality, Vikas.

B. CO-PO & CO-PSO MAPPING TABLE

MAPPING	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PS01	PS02	PS03
CO1	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO2	2	3	3	2	2	1	-	-	1	-	1	2	3	-	2
CO3	2	3	3	3	3	1	-	-	2	-	1	2	3	-	2
CO4	2	3	2	3	2	1	-	-	1	-	1	2	3	-	2
CO5	2	2	3	3	2	1	-	-	1	-	1	2	3	-	2

C.THEORY LECTURE PLAN

UNIT	CONTENT	NO. OF LECTURES
I	**Introductionto Software Engineering:** The Evolving Role of Software, Changing Nature of Software	1
	Software Engineering Discipline, Software Myths	1
	A Generic View of Process: Software Engineering- A Layered Technology, A Process Framework	1
	The Capability Maturity Model Integration (CMMI), Process Patterns, Process Assessment	1
	Personal andTeam Process Models	1
	Software Development Life Cycle (SDLC)Models: The Waterfall Model, Incremental Process Models, The Unified Process	2
	Evolutionary Process Models, Prototype Model	1
	Spiral Model, COCOMO Model	2

2.	**Software Requirements:** Functional and Non-Functional Requirements, User Requirements, System Requirements	1
	Interface Specification, The Software Requirements Document	1
	Requirements Engineering Process: Feasibility Studies, Requirements Elicitation and Analysis, Requirements Validation, Requirements Management	2
	Data Flow Diagrams, Entity Relationship Diagrams	1
	Software Quality Assurance (SQA): Verification and Validation, SQA Plans, Software Quality Frameworks	1
	ISO 9000 Models, SEI-CMM Model	2
3.	**Design Strategies:** Function Oriented Design, Object Oriented Design	1
	Top-Down and Bottom-Up Design	1
	Software Measurement and Metrics: Various Size Oriented Measures, Function Point (FP) Based Measures	1
	Cyclomatic Complexity Measures, Control Flow Graphs	1
4.	**Software Testing:** Testing Objectives, Unit Testing, Integration Testing	2
	Acceptance Testing, Regression Testing, Testing for Functionality and Testing for Performance	2
	Validation Testing, System Testing	2
	Top Down and Bottom- UpTesting Strategies: Test Drivers and Test Stubs, Structural Testing (White Box Testing)	1
	Functional Testing (Black Box Testing), Test Data Suit Preparation, Alpha and Beta Testing of Products	2
	Static Testing Strategies: Formal Technical Reviews (Peer Reviews), Walk Through, Code Inspection	1
	Compliance with Design and Coding Standards	2
5.	**Software Project Management:** Project Scheduling, Staffing	1
	Software Configuration Management, Quality Assurance, Project Monitoring	2
	Risk Management: Reactive Vs Proactive Risk Strategies, Risk Refinement	1
	Software Risks, Risk Identification, Risk Projection	1
	RMMM, RMMM Plan	1
TOTAL CLASSES		40

D. PRATICAL CLASS PLAN

1. An introduction to software engineering.
2. Preparation of Software Requirement Specification Document, Design Documents and TestingPhase related documents.
3. Development of Draw Level-0 and Level-1 Data Flow Diagram, data dictionary, E-R diagram, structured chart for the project.
4. To study and draw various UML diagrams.
5. Draw the use case diagram and specify the role of each of the actors. Also state the precondition,post condition and function of each use case.
6. To illustrate the use of class diagrams.
7. To draw an activity diagram and use case diagram for ATM and Library Management System.
8. Performing the Design by using any Design phase CASE tools.
9. Draw Object Diagram for ATM System.
10. Development of State Transition Diagram.
11. Draw ER Diagram for Hospital Management System.
12. Develop the prototype of the product.
13. Develop test cases for various white box and black box testing techniques.
14. Use one project management tool like Libra.
15. Preparation of Software Configuration Management and Risk Management related documents.

Course No	Title of the Course	Course Structure	Pre-Requisite
CICPC302	Database Management Systems	3L - 0T - 2P	None

COURSE OUTCOMES

At the end of the course students will be able to

- **CO 1 :** Understand fundamentals of Database Management Systems.
- **CO 2 :** Design database models and learn database languages to write queries to extract information from databases.
- **CO 3 :** Identify database anomalies and improving the design of DBMS using modern Database tools and languages
- **CO 4 :** Applying concept of transaction management and concurrency control for database management.
- **CO 5 :** Proposing and effective storage organization and database recovery mechanism.

A. Syllabus

UNIT	CONTENTS
1	**Introduction:** Database management system Characteristics of the Database, Database Systems and Architecture, Data Models, Schemes & Instances, DBMS Architecture & Data Independence, Database administrator & Database Users, Database Languages & Inter- faces, DDL, DML, DCL, Overview Relational Data Base Management Systems
2	**Data Modeling:** Data modeling using The Entity-Relationship Model Entities, Attributes and Relationships, Cardinality of Relationships, Strong and Weak Entity Sets, Generalization, Specialization, and Aggregation, Translating your ER Model into Relational Model, Relationships of higher degree.
3	**Relational Model, Languages & Systems**: Relational Data Model concepts, Relational Model Constraints, integrity constraints ,Keys domain constraints, referential integrity, assertions triggers, foreign key Relational Algebra and calculus, SQL, Database security. **Relational Data Base Design:** Functional Dependencies & Normalization for Relational Databases, Functional Dependencies, Normal Forms Based on Primary Keys, (1NF, 2NF, 3NF & BCNF), Lossless Join and Dependency Preserving Decomposition, Functional dependencies and its closure, covers and equivalence.
4	**Transaction Management:** Transaction Concept and State, Implementation of Atomicity and Durability, Concurrent Executions, Serializability: Testing of serializability, Serializability of schedules, conflict & view serializable schedule. **Concurrency Control Techniques:** Lock-Based Protocols, Timestamp-based Protocols, validation based protocol, Deadlock Handling
5	**Recovery System:** Recoverability: Failure Classification, Storage Structure, Recovery and Atomicity, Log-based Recovery, Shadow Paging, Recovery with Concurrent Transactions **Storage organization:** Indexing, Hashing, file storage.

SUGGESTED READINGS:

Text book:

1. Korth, Silbertz, Sudarshan, "Data base system concepts", McGraw-Hill, Seventh edition, 2019

Reference books

1. Elmasri, Navathe, "Fundamentals of Database systems", Pearson, seventh edition 2017.
2. Date C.J., "An Introduction to Database systems", Pearson India Eight edition,2004

15/9/24

B. CO-PO & CO-PSO MAPPING TABLE

MAPPING	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PS01	PS02	PS03
CO1	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO2	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO3	2	3	3	3	3	1	-	-	2	-	1	2	3	-	2
CO4	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO5	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2

C.THEORY LECTURE PLAN

Unit	Sub Topic	Total No. of Lectures
1	Overview. Database system Vs file system, History of DBMS	7
	Characteristics of the Database	
	Database system concept and architecture, Overall Database Structure	
	Data model schema and instances	
	Data independence	
	Database languages: DDL, DML, DCL	
	Database Users Data Definitions Language Keys, Concepts of Super Key, candidate key, primary key	
2	ER Model Concepts, Notation for ER Diagram, Mapping Constraints	6
	Extended ER Model Concept: Specialization, Generalization, Attribute Inheritance, Aggregation	
	Reduction of an ER Diagrams to Tables, Relationship of Higher Degree	
3	Relational Data Model Concepts	20
	Integrity Constraints, Entity Integrity, Referential Integrity, Keys Constraints, Domain Constraints	

Relational Algebra selection, projection, set operation
Cartesian product, Join, Extended RA operation
Characteristics of SQL, advantage of SQL. SQL data type and literals. Types of SQL Statements
DDL
TRC, DRC
Select queries, DML
Aggregate functions, group by and having clause
Join, Union, Intersection, Minus
Sub queries.
Views and indexes
Procedures in SQL/PL SQL
Cursors, Triggers
Functional dependencies, Functional dependencies- Closure of FDs, Armstrong's Axioms,
Closure of Attribute set
Closure of functional dependency
Canonical Cover
1st Normal Form and 2nd Normal form
3rd Normal from and BCNF
Preserving join dependency and lossless join decomposition
Questions on Normalization
Normalization using MVD, 4th Normal Form
Normalization using JD, 5th Normal Form

4	Introduction to transaction and transaction properties	7
	Serializability of schedules, Conflict serializable schedule	
	View serializable schedule	
	Testing of serializability	
	Introduction to Concurrency Control	
	Locking Techniques for Concurrency Control,	
	Time Stamping Protocols for Concurrency Control, Validation Based Protocol , Multiple Granularity, Multi Version Schemes, Checkpoints.	
	Deadlock handling	
5	Recoverability: Failure Classification, Storage Structure, Recovery and Atomicity, Log-based Recovery	7
	2-phase locking protocols	
	Log based recovery	
	Shadow Paging, Recovery with Concurrent Transactions	
	Storage organization: Indexing	
	Hashing, File storage	
	TOTAL CLASSES	47

D. PRACTICAL CLASS PLAN:

Following is only a suggestive list of experiments. For better coverage faculty may increase the list of experiments.

Q 1: Consider the following relational schema

SAILORS (sid, sname, rating, date_of_birth)
BOATS (bid, bname, color)
RESERVES (sid, bid, date, time slot)

Write the following queries in SQL and relational algebra

a) Find sailors who've reserved at least one boat
b) Find names of sailors who've reserved a red or a green boat in the month of March.
c) Find names of sailors who've reserved a red and a green boat
d) Find sid of sailors who have not reserved a boat after Jan 2018.
e) Find sailors whose rating is greater than that of all the sailors named "John"

f) Find sailors who've reserved all boats
g) Find name and age of the oldest sailor(s)
h) Find the age of the youngest sailor for each rating with at least 2 such sailors

Q2. Consider the following relational schema:

CUSTOMER (cust_num, cust_lname ,cust_fname, cust_balance);
PRODUCT (prod_num, prod_name, price)
INVOICE (inv_num, prod_num, cust_num, inv_date ,unit_sold, inv_amount);

Write SQL queries and relational algebraic expression for the following

a) Find the names of the customer who have purchased no item. Set default value of Cust_balance as 0 for such customers.
b) Write the trigger to update the CUST_BALANCE in the CUSTOMER table when a new invoicerecord is entered for the customer.
c) Find the customers who have purchased more than three units of a product on a day.
d) Write a query to illustrate Left Outer, Right Outer and Full Outer Join.
e) Count number of products sold on each date.
f) As soon as customer balance becomes greater than Rs. 100,000, copy the customer_num in new Tablecalled "GOLD_CUSTOMER"
g) Add a new attribute CUST_DOB in customer table

Q 3: Consider the following relational schema

DEPARTMENT (Department_ID, Name, Location_ID)
JOB (Job_ID , Function)
EMPLOYEE (Employee_ID, name, DOB, Job_ID ,Manager_ID, Hire_Date, Salary, department _id)

Answer the following queries using SQL and relational algebra:

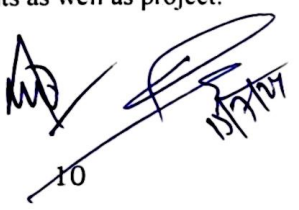
a) Write a query to count number of employees who joined in March 2015
b) Display the Nth highest salary drawing employee details.
c) Find the budget (total salary) of each department.
d) Find the department with maximum budget.
e) Create a view to show number of employees working in Delhi and update it automatically when the database is modified.
f) Write a trigger to ensure that no employee of age less than 25 can be inserted in the database.

Q4: PROJECT

Students are required to develop a DBMS for the applications assigned to them. Following items are required to be submitted for the project:

a) Problem Statement
b) ER model/ Relational Model
c) Integrity Constraints implemented
d) Suitable Queries to create and manage database

Note: Students have to make sure that they have defined proper integrity constraints to ensure consistency of database used in assignments as well as project.



Course No	Title of the Course	Course Structure	Pre-Requisite
CICPC303	Design and Analysis of Algorithms	3L - 0T - 2P	None

COURSE OUTCOMES

At the end of the course students will be able to

1. Analyze the asymptotic performance of algorithms.
2. Write rigorous correctness proofs for algorithms.
3. Apply demonstrate a familiarity with major algorithms and data structures.
4. Apply important algorithmic design paradigms and methods of analysis.
5. Implement sorting and searching schemes and analyze result with python programming language.

A. Syllabus

UNIT	CONTENTS
1	Introduction to algorithms, analyzing algorithms, Asymptotic notations and their significance, complexity analysis of algorithms, worst case and average case. Basic introduction to algorithmic paradigms like divide and conquer, recursion, greedy, dynamic programming etc.
2	Trees: Binary Tree, Binary search tree, AVL tree, red-black tree. Searching: Linear Search, Binary Search. Hashing, their applications. Priority queues, heaps. Sorting: comparison based sorting – Bubble sort, Insertion sort, selection sort, quick sort, heap sort, merge sort, their analysis.
3	**Divide and Conquer** with Examples: Sorting, Matrix Multiplication, Convex Hull and Searching. **Greedy Methods** with Examples: Knapsack, Minimum Spanning Trees – Prim's and Kruskal's Algorithms, Single Source Shortest Paths - Dijkstra's and Bellman Ford Algorithms, All pair shortest path algorithm: Floyd-Warshall algorithm. Advanced Topics: Amortized complexity analysis, Fibonacci heap.
4	Introduction to randomized algorithms, their types, Randomized Quicksort, Randomized Min-Cut, Applications of randomized algorithms. Application areas: Geo- metric algorithms: convex hulls, nearest neighbor, Voronoi diagram, etc.
5	Graph Algorithms: BFS, DFS, connected components, topological sort, network flows, matching, etc. Optimization techniques: linear programming Reducibility between problems and NP-completeness: discussion of different NP-complete problems like satisfiability, clique, vertex cover, independent set, Hamiltonian cycle, TSP, knapsack, set cover, bin packing, etc. Backtracking, branch and bound, Approximation algorithms: Constant ratio approximation algorithms.

REFERENCE BOOKS

1. E. Horowitz, S. Sahni, and S. Rajsekaran, "Funadmentals of Computer Algorithms," Galotia Publication
2. T .H . Cormen, C .E .Leiserson, R .L .Rivest "Introduction to Algorithms", PHI.
3. Sedgewich, Algorithms in C, Galgotia
4. Berman. Paul, "Algorithms, Cengage Learning".
5. Richard Neopolitan, Kumar SS Naimipour, "Foundations of Algorithms"
6. Aho, Hopcraft, Ullman, "The Design and Analysis of Computer Algorithms" Pearson Education, 2008.

B. CO-PO & CO-PSO MAPPING TABLE

MAPPING	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PS01	PS02	PS03
CO1	3	3	2	1	2	-	-	-	1	-	1	2	3	-	2
CO2	3	3	2	1	2	-	-	-	1	-	1	2	3	-	2
CO3	3	3	2	1	2	-	-	-	1	-	1	2	3	-	2
CO4	3	3	2	1	2	-	-	-	1	-	1	2	3	-	2
CO5	3	3	2	1	2	-	-	-	1	-	1	2	3	-	2

C.THEORY LECTURE PLAN

UNIT	CONTENTS	Total No. of Lectures
1	Introduction to algorithms	8
	Analyzing algorithms	
	Asymptotic notations and their significance	
	Complexity analysis of algorithms: Worst case	
	Complexity analysis of algorithms: Average case	
	Basic introduction to algorithmic paradigms: Divide and conquer	
	Basic introduction to algorithmic paradigms: Recursion	
	Basic introduction to algorithmic paradigms: Greedy and Dynamic Programming	
2	Binary Tree, Binary search tree	8
	AVL tree, Red-black tree	
	Searching: Linear Search, Binary Search	
	Hashing and their applications	
	Priority queues and heaps	
	Sorting: Bubble sort, Insertion sort and their analysis	
	Sorting: Selection sort, Quick sort and their analysis	
	Sorting: Heap sort, Merge sort and their analysis	
3	Divide and Conquer: Sorting, Matrix Multiplication	8
	Divide and Conquer: Convex Hull, Searching	
	Greedy Methods: Knapsack	
	Minimum Spanning Trees: Prim's Algorithm, Kruskal's Algorithm	
	Single Source Shortest Paths: Dijkstra's Algorithm, Bellman Ford Algorithm	
	All pair shortest path algorithm: Floyd-Warshall algorithm	
	Amortized complexity analysis	

15/7/24

4	Fibonacci heap	8
	Introduction to randomized algorithms	
	Types of randomized algorithms	
	Randomized Quicksort	
	Randomized Min-Cut	
	Applications of randomized algorithms	
	Geometric algorithms: Convex hulls	
	Geometric algorithms: Nearest neighbor	
	Geometric algorithms: Voronoi diagram	
5	Graph Algorithms: BFS, DFS, Connected components	8
	Graph Algorithms: Topological sort, Network flows and matching	
	Optimization techniques: Linear programming	
	Reducibility between problems and NP-completeness	
	Discussion of NP-complete problems: Satisfiability, Clique, Vertex cover, Independent set	
	Discussion of NP-complete problems: Hamiltonian cycle, TSP, Knapsack, Set cover, Bin packing	
	Backtracking and branch and bound	
	Approximation algorithms: Constant ratio approximation algorithms	
	TOTAL CLASSES	40

D. PRACTICAL CLASS PLAN:

Following is only a suggestive list of experiments. For better coverage faculty may increase the list of experiments.

1. Program for Recursive Binary & Linear Search.
2. Program for Selection Sort.
3. Program for Insertion Sort.
4. Program for Quick Sort.
5. Sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n > 5000 and record the time taken to sort. Plot a graph of the time taken versus n on graph sheet. The elements can be read from a file or can be generated using the random number generator. Demonstrate using Java how the divide-and-conquer method works along with its time complexity analysis: worst case, average case, and best case.
6. Program for Heap Sort.
7. Program for Merge Sort.
8. Sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n > 5000, and record the time taken to sort. Plot a graph of the time taken versus n on graph sheet. The elements can be read from a file or can be generated using the random number generator. Demonstrate how the divide-and-conquer method works along with its time complexity analysis: worst case, average case, and best case.
9. Find Minimum Spanning Tree using Kruskal's Algorithm.

10. Find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm. Use Union-Find algorithms in your program.
11. Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.
12. From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.
13. Knapsack Problem using Greedy Solution.
14. Implement the 0/1 Knapsack problem using (a) Dynamic Programming method (b) Greedy method.
15. Implement N Queen Problem using Backtracking.
16. Design and implement to find a subset of a given set S = {S1, S2, ..., Sn} of n positive integers whose SUM is equal to a given positive integer d. For example, if S = {1, 2, 5, 6, 8} and d = 9, there are two solutions {1, 2, 6} and {1, 8}. Display a suitable message if the given problem instance doesn't have a solution.
17. Implement Travelling Salesman Problem.
18. Write programs to (a) Implement All-Pairs Shortest Paths problem using Floyd's algorithm. (b) Implement Travelling Sales Person problem using Dynamic programming.
19. Design and implement to find all Hamiltonian Cycles in a connected undirected Graph G of n vertices using backtracking principle.

Course No	Title of the Course	Course Structure	Pre-Requisite
CICPC304	Computer Architecture and Organization	3L - 1T - 0P	---

COURSE OUTCOMES

At the end of the course students will be able to

1. Understand the architecture of modern processors and organization of its components, and relationship between hardware and software in digital machines.
2. Design instructions and corresponding logic circuits for a simple CPU with its essential components such as ALU, a register file, memory and input-output.
3. Understand the organization of computer systems
4. Understand the computation standards and using them in writing algorithms
5. Appreciate the evolving technology that governs the evolution of modern computers and continue to keep abreast of state-of-art in computing technology

A. Syllabus

UNIT	CONTENTS
1	**Overview of computer organization**: Characteristics of a general purpose computer. The stored program concept, von Neumann architecture, Harvard architecture. Programmer's model - the Instruction set architecture (ISA). ISA design and performance criteria. Basic computer organization with CPU, memory and IO subsystems. Interconnect busses, Evolution of CISC and RISC based processors and their merging.
2	**Instruction Set Architectures**: Machine instruction, Machine cycle and Instruction cycles, Instruction Set: memory and non-memory reference instructions, instruction categories: data movement, data manipulation, program control and machine control instructions, CISC types addressing modes and instruction formats. RISC type addressing modes and instruction formats.
3	**Central Processing Unit**: Specification of a simple CPU using RTL. Design of the data path for the simple CPU. Designing the hardwired control path for the simple CPU. Performance analysis of the simple CPU. Enhancement of the ISA for the simple CPU and design extensions. Characteristics of RISC CPU design: ISA characteristics, pipelining, data and instruction caches. Practical case studies in CISC type and RISC type CPU designs
4	**Micro programmed Control Unit**: Control memory system, Microinstruction-sequencing, conditional branch, mapping and subroutines, direct, horizontal and vertical mi- crocoding, micro-instruction format and symbolic representation, design of micro-control unit for a simple CPU, applications of microprogramming **Memory organization**: Memory hierarchy, Cache organization: Direct, associative and Set associative cache. Auxiliary memory organization, RAID organizations **Input output organization:** IO interfacing, Asynchronous data transfer, Programmed IO, Interrupt driven IO. Priority schemes, Direct Memory Access, Serial communication technique
5	**Computer arithmetic**: Design of Binary addition and subtraction units. Algorithms for multiplication and division and their implementation, Floating point arithmetic, etc. **Pipelined architecture**: Basic concepts of pipelining, Speedup and throughput, Minimum Average Latency, Instruction pipeline. **GPU architecture**: Hardware Basics, Execution Model, GPU instruction set architecture, NVIDIA GPU instruction set architecture **Guidelines for Project work:** Exercises using assembly-level programming and debugging to illustrate the working of instructions in the ISA of a CISC based /RISC based processor. These exercises should illustrate the status of various registers, flags, counters and pointers after data move- ment, data manipulation, program control, and stack operations. - Semester-long group project on the design and simulation /hardware emulation of a simple processor.

SUGGESTED READINGS:

Text book:

M. Morris Mano, "Computer System Architecture" , PHI

Reference books

1. William stallings, "Computer Organization and Architecture, PHI" 2. M. Morris Mano,
2. J.D. Carpinelli, "Computer Systems Organization and Architecture," Pearson Education
3. Heuring and Jordan, Pearson Education, "Computer Systems Design and Architecture"
4. Tor M. Aamodt, Wilson Wai Lun Fung, Timothy G. Rogers General-Purpose Graphics Processor Architectures

B. CO-PO & CO-PSO MAPPING TABLE

MAPPING	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3	PSO4
CO1	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO2	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO3	2	3	3	3	3	1	-	-	2	-	1	2	3	-	2
CO4	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2
CO5	2	3	3	3	2	1	-	-	1	-	1	2	3	-	2

C. THEORY LECTURE PLAN

Unit	Sub Topic	Total No. of Lectures
1	Overview of computer organization	8
	Characteristics of a general purpose computer	
	The stored program concept, von Neumann architecture	
	Harvard architecture	
	Programmer's model - the Instruction set architecture (ISA)	
	ISA design and performance criteria , Basic computer organization with CPU	
	memory and IO subsystems, Interconnect busses	
	Evolution of CISC and RISC based processors and their merging	
2	Instruction Set Architectures	14
	Machine instruction	
	Machine cycle	
	Instruction cycles	
	Instruction Set: memory and non-memory reference instructions	
	instruction categories: data movement	
	data manipulation	
	program control	
	machine control instructions	
	Hardware Interfacing	

16 15/7/24

	CISC types addressing modes	
	CISC types instruction formats	
	RISC type addressing modes	
	RISC type instruction formats	
3	Overview of Central Processing Unit	10
	Specification of a simple CPU using RTL	
	Design of the data path for the simple CPU	
	Designing the hardwired control path for the simple CPU	
	Performance analysis of the simple CPU	
	Enhancement of the ISA for the simple CPU and design extensions	
	Debugging and Profiling Extensions	
	Characteristics of RISC CPU design: ISA characteristics	
	pipelining, data and instruction caches	
	Practical case studies in CISC type and RISC type CPU designs	
4	Micro programmed Control Unit: Control memory system, Microinstruction-sequencing, conditional branch, mapping and subroutines	8
	direct, horizontal and vertical mi- crocoding,micro-instruction format and symbolic representation	
	design of micro-control unit for a simple CPU, applications of microprogramming	
	Memory organization: Memory hierarchy, Cache organization: Direct, associative and Set associative cache	
	Auxiliary memory organization, RAID organizations	
	Input output organization: IO interfacing, Asynchronous data transfer	
	Programmed IO, Interrupt driven IO, Priority schemes	
	Direct Memory Access, Serial communication technique	
5	Computer arithmetic: Design of Binary addition and subtraction units	8
	Algorithms for multiplication and division and their implementation, Floating point arithmetic, etc.	
	Pipelined architecture: Basic concepts of pipelining	
	Speedup and throughput	
	Minimum Average Latency, Instruction pipeline.	
	GPU architecture: Hardware Basics	
	Execution Model, GPU instruction set architecture	
	NVIDIA GPU instruction set architecture	
	TOTAL CLASSES	48

Course No	Title of the Course	Course Structure	Pre-Requisite
CIEPC305	Microprocessor and Microcontroller	3L-0T-2P	None

COURSE OUTCOMES (COs)

After completion of this course, the students are expected to be able to demonstrate the following knowledge, skills and attitudes:

1. Acquire knowledge of internal architecture and configuration of 8-bit microprocessors.
2. Develop programming skills to create programs using assembly language programming.
3. Understand the salient features of the x86 architecture and its salient features.
4. Acquire hands-on knowledge of interfacing microprocessors with peripherals.
5. Understand the architecture and working of microcontrollers and their utility.
6. Acquire introductory knowledge about high-end microprocessors and microcontrollers.

A. Syllabus

COURSE CONTENT

Unit 1

Intel 8085 microprocessor: Basic concepts of microprocessor, microcomputer, microcontroller. 8085 microprocessor Core architecture - Various registers- Bus Timings. Multiplexing and De-multiplexing of Address Bus. Decoding and Execution. PIN diagram, addressing modes. Vectored, non- vectored interrupts, Hardware and software interrupts and interrupt handling of 8085. Concept of stack and stack handling. Instruction set (instruction format, opcode, mnemonic), subroutines, timing diagrams and t-states of different instructions, programming.

Unit 2

Intel 8086 microprocessor: Architecture (pins, bus interface unit, execution unit, register set, pipelining), memory addressing, segmentation, instruction set (data transfer, arithmetic, logic, string, long and short control transfer and processor control). Addressing modes, programming, assemblers, parameter passing to subroutines, hardware and software interrupts and interrupt handling of 8086.

Unit 3

Interfacing a microprocessor with RAM and ROM chips, address allocation and decoding techniques. Interfacing a microprocessor with RAM and ROM chips, address allocation and decoding techniques. Memory-mapped i/o. Interfacing with 8255 programmable peripheral interfaces (architecture, ports, i/o modes and BSR mode). Basic architecture and features of 8254 programmable timer, 8257 programmable DMA con- troller, 8259 programmable interrupt controller, 8279 programmable keyboard and dis- play controller.

Unit 4

Microcontrollers: 8051 microcontroller: architecture, i/o ports, memory organization, addressing modes, instruction set, simple programs. Introduction to IoT: basic architecture, sensing and actuating, application domains.

Unit 5

High-end microprocessors and microcontrollers: Important features of 32-bit processors, RISC and Pentium Implementation of memory management schemes like segmentation, paging and virtual memory at the Hardware level. Introduction to Arduino: basic architecture, simple programs.

Recommended books:

1. Ramesh S. Gaonkar, "Microprocessor Architecture, Programming, and Applications with the 8085" Prentice Hall.
2. D. V. Hall, "Microprocessor and Interfacing Programming & Hardware" TMH – 2nd Edition.
3. S. P. Morse, "8086 Primer: An Introduction to Its Architecture, System Design and Programming" Hayden Book Co.
4. S. Monk, "Programming Arduino: Getting Started with Sketches", 2nd Edition, McGraw-Hill.
5. M.A. Mazidi et. al. "The 8051 Microcontroller and Embedded Systems: Using Assem- bly and C" Pearson Publishers.

B. CO-PO & CO-PSO MAPPING TABLE

Mapping	PO 1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	2	1									1	2	2	2
CO2	3	3	1	2	2							1	3	3	2
CO3	3	3	2	1	2							1	2	3	3
CO4	3	3	1	2	2							1	2	2	3
CO5	3	3	1	2	1						2	1	2	3	3
CO6	3	2	1	2	2						2	1	3	2	3

C. THEORY LECTURE PLAN

S.No.	CONTENT	NO. OF LECTURES
	UNIT-I	
1	Basic concepts of microprocessor, microcomputer, microcontroller. , programming.	1
2	8085 microprocessor – Core architecture - Various registers	1
3	PIN diagram	1
4	Bus Timings, Multiplexing and De-multiplexing of Address Bus, Decoding and Execution	1
5	Vectored, non- vectored interrupts, Hardware and software interrupts and interrupt handling of 8085.	2
6	Addressing modes.	1

7	Instruction set (instruction format, opcode, mnemonic), Programming	2
8	Concept of stack and stack handling, subroutines.	1
9	Timing diagrams and t-states of different instructions	1
	UNIT-II	
10	8086 microprocessor Architecture (pins, bus interface unit, execution unit, register set, pipelining),	2
11	memory addressing, segmentation	1
12	Instruction set (data transfer, arithmetic, logic, string, long and short control transfer and processor control),	2
13	Programming concepts	2
14	Addressing modes, Assemblers, parameter passing to subroutines	1
15	Hardware and software interrupts and interrupt handling of 8086,	1
	UNIT-III	
16	Interfacing a microprocessor with RAM and ROM chips,	1
17	Address allocation and decoding techniques. Memory-mapped i/o,	1
18	Interfacing with 8255 programmable peripheral interfaces (architecture, ports, i/o modes and BSR mode),	2
19	Basic architecture and features of 8254 programmable timer,	2
20	8257 programmable DMA con- troller	1
21	8259 programmable interrupt controller	1
22	8279 programmable keyboard and dis- play controller,	1
	UNIT-IV	
23	8051 microcontroller: Architecture	1
24	I/o ports, memory organization	1
25	Addressing modes	1
26	Instruction set, simple programs	2
27	Introduction to IoT: basic architecture, sensing and actuating, application domains,	1
	UNIT-V	

28	Important features of 32-bit processors	1
29	RISC and Pentium	1
30	Implementation of memory management schemes like segmentation, paging and virtual memory at the Hardware level	1
31	Introduction to Arduino: basic architecture	1
32	Simple programs.	1
TOTAL CLASSES		40

D. PRACTICAL CLASS PLAN

Suggested List of Experiments

1. Write a program to add two 16-bit numbers with/without carry using 8086 microprocessor.
2. Write a program to subtract two 16-bit numbers with/without borrow using 8086 microprocessor.
3. Write a program to multiply two 8-bit numbers by repetitive addition using 8086.
4. Write a program to generate Fibonacci series up to 16 terms.
5. Write a program to sort an array of data in ascending order.
6. Write a program to find the largest/smallest number from a array of data.
7. Write a program to generate factorial of a number from 0 to 9.
8. Write a program to read 16 bit data from a port and display the same in another port.
9. Write a program to generate square wave of 10 KHz using timer 1 in mode 1(using 8051 microcontroller).
10. Write a program to transfer data from ROM memory to RAM memory.
11. Write a program for the traffic light controller using 8086 microprocessor.